

Методическая разработка: создание учебного CA и TLS-сертификата для web-сервера

1. Паспорт работы

Тема лекции: OpenSSL, CA и TLS-сертификаты для серверных служб.

Учебный сценарий: создать учебный root CA, выпустить сертификат для web-сервера web01.company.local, подключить HTTPS в Nginx, импортировать сертификат CA на клиент и проверить доверие.

ОС: Debian 12 Server или Ubuntu Server, Linux/Windows Client.

ПО: OpenSSL, Nginx, curl, браузер, update-ca-certificates.

Формат: демонстрация преподавателя и самостоятельное повторение студентами.

Время: 2-4 академических часа.

Предварительная подготовка: на web01 установлен Nginx и опубликована страница из первой лекции.

Итоговый результат: есть учебный CA, web-сервер работает по HTTPS, клиент доверяет сертификату после импорта CA.

2. Что будет настроено

- закрытый ключ root CA;
- самоподписанный сертификат root CA;
- закрытый ключ web-сервера;
- CSR для web01.company.local;
- TLS-сертификат web-сервера с SAN;
- HTTPS-конфигурация Nginx;
- правило firewall для 443/tcp;
- проверка через openssl s_client, curl и браузер;
- импорт CA на клиентскую машину.

3. Схема учебного стенда

```
ca01
Root CA
|
| подписывает CSR
v
web01.company.local <----- HTTPS -----> client01
192.168.10.20      192.168.10.101
```

Узел	Роль	IP-адрес	Назначение
ca01	Учебный центр сертификации	192.168.10.70	Выпуск сертификатов
web01	Web-сервер	192.168.10.20	HTTPS-сервис
client01	Клиент	192.168.10.101	Проверка доверия

Для упрощённой лабораторной CA можно создать на web01, но в отчёте нужно указать, что в реальной инфраструктуре ключ CA хранят отдельно и защищают строже, чем обычный сервер.

4. Исходные условия и правила безопасности

Закрытый ключ — это секрет. Если злоумышленник получит ключ CA, он сможет выпускать доверенные сертификаты от имени организации.

Перед началом:

- сделать снимок VM;
- проверить имя и IP-адрес web01;
- убедиться, что Nginx работает по HTTP;
- подготовить каталог для CA;
- не передавать закрытые ключи через чат, почту или общий каталог.

Проверка Nginx:

```
systemctl status nginx
curl -I http://192.168.10.20
```

Первая команда проверяет службу, вторая — HTTP-ответ web-сервера.

5. Подготовка каталогов CA

Создать рабочий каталог:

```
mkdir -p ~/pki/{ca,server,certs}
cd ~/pki
```

Что делают команды:

- `mkdir -p` создаёт структуру каталогов;
- `ca` будет хранить ключ и сертификат CA;
- `server` будет хранить ключ и CSR web-сервера;
- `certs` будет хранить выпущенные сертификаты.

Проверка:

```
ls -R ~/pki
```

Ожидаемый результат: видны каталоги `ca`, `server`, `certs`.

6. Создание root CA

6.1. Закрытый ключ root CA

```
openssl genrsa -out ~/pki/ca/company-root-ca.key 4096
```

Что делает команда:

- создаёт RSA-ключ длиной 4096 бит;
- сохраняет закрытый ключ CA в файл `company-root-ca.key`.

Ограничить права:

```
chmod 600 ~/pki/ca/company-root-ca.key
```

Команда разрешает читать и изменять ключ только владельцу файла.

Проверка:

```
ls -l ~/pki/ca/company-root-ca.key
```

Ожидаемый результат: права начинаются с `-rw-----`.

6.2. Самоподписанный сертификат root CA

```
openssl req -x509 -new -nodes \  
-key ~/pki/ca/company-root-ca.key \  
-sha256 -days 3650 \  
-out ~/pki/ca/company-root-ca.crt \  
-subj "/C=KZ/O=Company Lab/CN=Company Lab Root CA"
```

Что делает команда:

- req -x509 создаёт самоподписанный сертификат;
- -key указывает закрытый ключ CA;
- -sha256 задаёт алгоритм хэширования;
- -days 3650 задаёт срок действия;
- -subj задаёт поля владельца сертификата.

Проверка сертификата:

```
openssl x509 -in ~/pki/ca/company-root-ca.crt -noout -subject -issuer -dates
```

Ожидаемый результат:

- subject и issuer совпадают;
- это признак самоподписанного root CA.

7. Создание ключа и CSR web-сервера

7.1. Закрытый ключ web-сервера

```
openssl genrsa -out ~/pki/server/web01.key 2048
```

Команда создаёт закрытый ключ web-сервера. Этот ключ остаётся на web01 и не должен передаваться клиентам.

Ограничить права:

```
chmod 600 ~/pki/server/web01.key
```

7.2. Конфигурация CSR с SAN

Современные браузеры проверяют Subject Alternative Name. Одного CN обычно недостаточно.

Создать файл:

```
nano ~/pki/server/web01-openssl.cnf
```

Содержимое:

```
[ req ]  
default_bits      = 2048  
prompt           = no  
default_md        = sha256  
distinguished_name = dn  
req_extensions    = req_ext  
  
[ dn ]  
C  = KZ  
O  = Company Lab  
CN = web01.company.local  
  
[ req_ext ]
```

```
subjectAltName = @alt_names

[ alt_names ]
DNS.1 = web01.company.local
DNS.2 = web01
IP.1 = 192.168.10.20
```

Что важно:

- CN задаёт основное имя;
- subjectAltName перечисляет имена и IP, по которым клиент будет открывать сервер;
- если клиент открывает `https://192.168.10.20`, IP должен быть в SAN.

7.3. Создание CSR

```
openssl req -new \
-key ~/pki/server/web01.key \
-out ~/pki/server/web01.csr \
-config ~/pki/server/web01-openssl.cnf
```

Что делает команда:

- создаёт Certificate Signing Request;
- включает в CSR данные из конфигурации;
- не содержит закрытый ключ.

Проверка CSR:

```
openssl req -in ~/pki/server/web01.csr -noout -subject -text | grep -A 5 "Subject Alternative Na
```

Ожидаемый результат: видны `DNS:web01.company.local` и `IP Address:192.168.10.20`.

8. Подписание сертификата web-сервера

```
openssl x509 -req \
-in ~/pki/server/web01.csr \
-CA ~/pki/ca/company-root-ca.crt \
-CAkey ~/pki/ca/company-root-ca.key \
-CAcreateserial \
-out ~/pki/certs/web01.crt \
-days 825 -sha256 \
-extensions req_ext \
-extfile ~/pki/server/web01-openssl.cnf
```

Что делает команда:

- подписывает CSR web-сервера сертификатом root CA;
- создаёт сертификат `web01.crt`;
- переносит SAN из конфигурации;
- создаёт serial-файл CA при первом выпуске.

Проверка:

```
openssl x509 -in ~/pki/certs/web01.crt -noout -subject -issuer -dates
```

Ожидаемый результат:

- subject содержит `web01.company.local`;
- issuer содержит `Company Lab Root CA`.

Проверка цепочки:

```
openssl verify -CAfile ~/pki/ca/company-root-ca.crt ~/pki/certs/web01.crt
```

Ожидаемый результат:

```
web01.crt: OK
```

9. Подключение HTTPS в Nginx

Скопировать сертификаты:

```
sudo mkdir -p /etc/nginx/ssl
sudo cp ~/pki/certs/web01.crt /etc/nginx/ssl/web01.crt
sudo cp ~/pki/server/web01.key /etc/nginx/ssl/web01.key
sudo chmod 600 /etc/nginx/ssl/web01.key
```

Что делают команды:

- создают каталог для TLS-файлов;
- копируют сертификат и ключ;
- ограничивают доступ к закрытому ключу.

Создать конфигурацию сайта:

```
sudo nano /etc/nginx/sites-available/web01-ssl
```

Содержимое:

```
server {
    listen 443 ssl;
    server_name web01.company.local web01 192.168.10.20;

    ssl_certificate      /etc/nginx/ssl/web01.crt;
    ssl_certificate_key  /etc/nginx/ssl/web01.key;

    root /var/www/html;
    index index.html;
}
```

Включить сайт:

```
sudo ln -s /etc/nginx/sites-available/web01-ssl /etc/nginx/sites-enabled/web01-ssl
```

Проверить синтаксис:

```
sudo nginx -t
```

Ожидаемый результат:

```
syntax is ok
test is successful
```

Перезагрузить Nginx:

```
sudo systemctl reload nginx
```

10. Firewall и проверка HTTPS

Разрешить HTTPS:

```
sudo ufw allow 443/tcp
```

Команда открывает входящие подключения к HTTPS.

Проверка firewall:

```
sudo ufw status numbered
```

Ожидаемый результат: есть правило 443/tcp ALLOW.

Проверка порта:

```
sudo ss -tulpen | grep ':443'
```

Ожидаемый результат: Nginx слушает 443/tcp.

Проверка TLS:

```
openssl s_client -connect 192.168.10.20:443 -servername web01.company.local
```

Что делает команда:

- устанавливает TLS-соединение;
- показывает сертификат сервера;
- показывает цепочку и ошибки доверия.

До импорта CA ожидаема ошибка доверия, потому что клиент ещё не доверяет учебному CA.

11. Импорт CA на Linux-клиент

Скопировать company-root-ca.crt на клиент и выполнить:

```
sudo cp company-root-ca.crt /usr/local/share/ca-certificates/company-root-ca.crt
sudo update-ca-certificates
```

Что делают команды:

- помещают сертификат CA в системное хранилище;
- обновляют список доверенных CA.

Проверка через curl:

```
curl -I https://web01.company.local
```

Ожидаемый результат:

```
HTTP/1.1 200 OK
```

Если имя не разрешается, временно добавить в /etc/hosts:

```
192.168.10.20 web01.company.local web01
```

12. Импорт CA на Windows-клиент

На Windows:

1. Скопировать файл company-root-ca.crt.
2. Открыть mmc.
3. Добавить оснастку Certificates для Local Computer.
4. Импортировать CA в Trusted Root Certification Authorities.
5. Открыть https://web01.company.local в браузере.

Ожидаемый результат: браузер показывает доверенное HTTPS-соединение без предупреждения о неизвестном издателе.

13. Диагностика и типовые ошибки

Симптом	Возможная причина	Проверка	Исправление
Nginx не запускается	Неверный путь к ключу или сертификату	<code>sudo nginx -t</code>	Исправить пути
Браузер не доверяет сертификату	CA не импортирован	Проверить хранилище CA	Импортировать root CA
Ошибка имени сертификата	Нет нужного DNS/IP в SAN	<code>openssl x509 -text</code>	Выпустить новый сертификат с SAN
HTTPS-порт закрыт	Firewall блокирует 443	<code>ufw status, ss</code>	Разрешить 443/tcp
curl не находит имя	Нет DNS или hosts-записи	<code>getent hosts</code>	Добавить DNS/hosts
Закрытый ключ читается всеми	Неверные права	<code>ls -l web01.key</code>	Установить <code>chmod 600</code>

14. Итоговая самостоятельная проверка

- ☐ Создан ключ root CA.
- ☐ Создан сертификат root CA.
- ☐ Создан ключ web-сервера.
- ☐ Создан CSR с SAN.
- ☐ Выпущен сертификат web-сервера.
- ☐ `openssl verify` возвращает OK.
- ☐ Nginx слушает 443/tcp.
- ☐ HTTPS открывается с клиента.
- ☐ После импорта CA браузер доверяет сертификату.

Финальные команды:

```
openssl x509 -in ~/pki/certs/web01.crt -noout -subject -issuer -dates
openssl verify -CAfile ~/pki/ca/company-root-ca.crt ~/pki/certs/web01.crt
sudo nginx -t
sudo ss -tulpen | grep ':443'
curl -I https://web01.company.local
```

15. Отчёт по работе

В отчёт включить:

1. Схему PKI.
2. Subject и issuer root CA.
3. Subject и SAN сертификата web-сервера.
4. Конфигурацию Nginx для HTTPS.
5. Результат `openssl verify`.
6. Результат `curl -I`.
7. Скриншот доверенного HTTPS в браузере.
8. Описание одной ошибки TLS и её исправления.

16. Контрольные вопросы

1. Для чего нужен TLS?
2. Почему закрытый ключ нельзя передавать клиенту?
3. Что такое CSR?
4. Чем root CA отличается от сертификата сервера?
5. Для чего нужен SAN?
6. Почему клиент должен доверять CA?

7. Как проверить сертификат через OpenSSL?
8. Что будет, если имя в URL не совпадает с SAN?

17. Итог

После выполнения методички студент должен понимать цепочку: закрытый ключ, CSR, CA, сертификат, настройка сервера, доверие клиента. HTTPS считается настроенным только после проверки сертификата, порта, web-службы и доверия на клиентской машине.